Round 2 Student Challenge Brief

AI Visual Quality Inspection using Python Full Stack

Your challenge

A manufacturing organization wants to screen product images for visible defects and let supervisors verify the results. Build a web application that runs the supplied Computer Vision classifier, records each inspection, processes image batches in the background, and preserves supervisor reviews.

Item	Requirement
Team size	3 or 4 students
Briefing	Day 1, 10:45 PM to 11:30 PM
Development	11:45 AM to 6:45 AM; 7 hours
Submission deadline	7:00 AM on Day 2
Evaluation	Day 2, 7:15 AM to 10:15 AM
Shortlist announcement	Day 2, 10:30 AM

Expected user workflow

- Administrator registers the supported product P-PLATE01.
- Engineer selects the product and uploads a JPG or PNG. The application shows the image, predicted class, and model confidence score.
- Engineer uploads a small batch and follows each item through queued, processing, completed, or failed status.
- Supervisor filters the saved inspection history, then confirms or corrects a prediction with an optional comment.

Scope and independent work

Use one product type and two classes: Normal and Defective. The supplied images are synthetic metal-plate illustrations, with scratches or pits used as visible defect examples. This is a software integration challenge, not evidence of real factory performance. No training, bounding boxes, live camera feed, or paid inference API is required.

You may reuse Round 1 login, database configuration, API setup, and interface layout. Build the new inspection workflow during Round 2. Mentors clarify requirements and common environment issues; teams design and implement their own solution.

Functional Requirements

- Use HTML, CSS, JavaScript, and Bootstrap with Django, Django REST Framework, and PostgreSQL or MySQL. Django admin is acceptable for product administration.
- Enforce roles in the backend. Only Administrators manage products. Engineers submit images and view their own inspection history. Supervisors view all inspections and submit reviews. An Engineer must not review results by calling the API directly.
- Validate actual file content, dimensions, and size before inference. Accept decoded JPEG or PNG only, at most 5 MiB per file, with each dimension from 64 to 4096 pixels inclusive. A batch contains 1 to 10 files. Reject empty or oversized batches.
- Apply the supplied preprocessing and frozen classifier. Show Normal or Defective and the confidence of the predicted class. Do not infer labels from filenames, folder names, CSV records, or lookup tables.
- Save product, image reference, original predicted class, confidence, defect probability, model version, submitting user, and timestamp. Keep these records after a page refresh or application restart.
- Use Celery and Redis for batch inference. Record per-item status and readable failure reasons. A bad file must not stop valid batch items. Persist invalid items as failed attempts without a prediction; never pass them to the model.
- Show history filters for prediction and review status. Supervisors confirm or correct completed inspections. Save reviewed class, reviewer, review timestamp, and optional comment separately from the immutable original model output.
- Write automated tests for invalid image rejection, supervisor-only review, and result persistence. Report a confusion matrix and defect recall using all 60 labeled samples.

Input pack

Folder or file	Purpose
Model/	Frozen model, reference inference helper, runtime dependencies, and metadata
Samples/	60 labeled sample images and labels.csv
Validation/	Three intentional invalid inputs for upload validation
product_details.csv	Supported product registration metadata
Round2_Model_and_Data_Guide	Preprocessing, model scope, file conventions, and metric definitions

The inference helper supplies only model loading and image preprocessing. Your team implements authentication, endpoints, storage, validation, task processing, screens, and reviews. Do not modify the model weights, feature extraction, or 0.5 decision threshold for the scored submission.

Submission Demonstration and Scoring

Submit by 7 AM on Day 2

Demonstration checklist

- Show product administration and a single image inspection with saved prediction and confidence.
- Submit a mixed batch containing valid and invalid files. Show background status changes, successful items, and readable failures.
- Filter history and correct a prediction as Supervisor. Show both the original output and review, then reload the record.
- Attempt a review as Engineer and show server-side denial.
- Explain the confusion matrix, defect recall, and at least one limitation. Judges may rename files and use unseen images with the same product framing.

Practice Example:

- While signed in as Engineer, attempt to confirm or correct a completed inspection using the review action or an accessible review-page URL.
- **Expected outcome:** The review is denied and no review or original prediction changes. The same permitted review succeeds as Supervisor. If no action can be exercised, record the limited evidence rather than claiming full authorization verification.